Training and generalization in overparameterized neural networks

Stage 1 review

Shreyas Kalvankar

January 28, 2026

TU Delft

Agenda

Agenda

1. Introduction & Problem setup
2. Related work: NTK, mean-field, spectral bias
3. Research Questions & Objectives
4. Preliminary observations
5. Planned work & Discussions

Introduction & Problem setup

Problem setup: Empirical Risk Minimization

- Given a parametric model $f_\theta : \mathbb{R}^d \to \mathbb{R}$ and a dataset $\{(x_i, y_i)\}_{i=1}^n$, with inputs $x_i \in \mathbb{R}^d$ and targets $y_i \in \mathbb{R}$ (stacked as $X \in \mathbb{R}^{d \times n}, y \in \mathbb{R}^n$).

Introduction & Problem setup

Problem setup: Empirical Risk Minimization

- Given a parametric model $f_\theta : \mathbb{R}^d \to \mathbb{R}$ and a dataset $\{(x_i, y_i)\}_{i=1}^n$, with inputs $x_i \in \mathbb{R}^d$ and targets $y_i \in \mathbb{R}$ (stacked as $X \in \mathbb{R}^{d \times n}, y \in \mathbb{R}^n$).
- Task: Find the parameters θ

Introduction & Problem setup

Problem setup: Empirical Risk Minimization

- Given a parametric model $f_\theta : \mathbb{R}^d \to \mathbb{R}$ and a dataset $\{(x_i, y_i)\}_{i=1}^n$, with inputs $x_i \in \mathbb{R}^d$ and targets $y_i \in \mathbb{R}$ (stacked as $X \in \mathbb{R}^{d \times n}, y \in \mathbb{R}^n$).
- Task: Find the parameters θ
- Objective: that minimizes the empirical risk:

Introduction & Problem setup

Problem setup: Empirical Risk Minimization

- Given a parametric model $f_\theta : \mathbb{R}^d \to \mathbb{R}$ and a dataset $\{(x_i, y_i)\}_{i=1}^n$, with inputs $x_i \in \mathbb{R}^d$ and targets $y_i \in \mathbb{R}$ (stacked as $X \in \mathbb{R}^{d \times n}, y \in \mathbb{R}^n$).
- Task: Find the parameters θ
- Objective: that minimizes the empirical risk: $\mathcal{L}(\theta) = \sum_{i=1}^{n} d(f_\theta(x_i), y_i) = \sum_{i=1}^{n} (f_\theta(x_i) - y_i)^2$

Large family of these models is generally trained using algorithms like gradient descent which is notoriously hard to analyse. To motivate the obstacles to deep learning theory, we begin with a simple case.

Introduction & Problem setup

Problem setup: Empirical Risk Minimization

- Given a parametric model $f_\theta : \mathbb{R}^d \to \mathbb{R}$ and a dataset $\{(x_i, y_i)\}_{i=1}^n$, with inputs $x_i \in \mathbb{R}^d$ and targets $y_i \in \mathbb{R}$ (stacked as $X \in \mathbb{R}^{d \times n}, y \in \mathbb{R}^n$).
- Task: Find the parameters θ
- Objective: that minimizes the empirical risk: $\mathcal{L}(\theta) = d(f_\theta(X), Y) = \|(f_\theta(X) - y)\|^2$

Large family of these models is generally trained using algorithms like gradient descent which is notoriously hard to analyse. To motivate the obstacles to deep learning theory, we begin with a simple case.

Introduction & Problem setup

Linear Networks

- *One-layer (no hidden layers):* $f_w(x) = x^\top w$, $w \in \mathbb{R}^d$

Introduction & Problem setup

Linear Networks

- *One-layer (no hidden layers):* $f_w(x) = x^\top w$, $w \in \mathbb{R}^d$
- Objective: $\mathcal{L}(w) = \frac{1}{2}\|X^\top w - y\|^2$
- Gradient flow: $\dot{w}_t = -\nabla_w \mathcal{L}(w_t) = -X(X^\top w_t - y) = Xy - XX^\top w_t$

Introduction & Problem setup

Linear Networks

- *One-layer (no hidden layers):* $f_w(x) = x^\top w$, $w \in \mathbb{R}^d$
- Objective: $\mathcal{L}(w) = \frac{1}{2}\|X^\top w - y\|^2$
- Gradient flow: $\dot{w}_t = -\nabla_w \mathcal{L}(w_t) = -X(X^\top w_t - y) = Xy - XX^\top w_t$
- Gradient descent is a linear dynamical system governed by the data covariance matrix.

Introduction & Problem setup

Linear Networks

- *One-layer (no hidden layers):* $f_w(x) = x^\top w$, $w \in \mathbb{R}^d$
- Objective: $\mathcal{L}(w) = \frac{1}{2}\|X^\top w - y\|^2$
- Gradient flow: $\dot{w}_t = -\nabla_w \mathcal{L}(w_t) = -X(X^\top w_t - y) = Xy - XX^\top w_t$
- Gradient descent is a linear dynamical system governed by the data covariance matrix.
- Parameter dynamics known, can be characterized and analysed.

Introduction & Problem setup

Linear Networks

- *One-layer (no hidden layers):* $f_w(x) = x^\top w$, $w \in \mathbb{R}^d$
- Objective: $\mathcal{L}(w) = \frac{1}{2}\|X^\top w - y\|^2$
- Gradient flow: $\dot{w}_t = -\nabla_w \mathcal{L}(w_t) = -X(X^\top w_t - y) = Xy - XX^\top w_t$
- Gradient descent is a linear dynamical system governed by the data covariance matrix.
- Parameter dynamics known, can be characterized and analysed.

This is the extent to which we can do deep theoretical analysis of the training dynamics.

Introduction & Problem setup

Linear Networks

- *Deep Linear (1 hidden layer of width m):* $f_{v,W}(x) := (Wx)^\top v$ where $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$.

Introduction & Problem setup

Linear Networks

- *Deep Linear (1 hidden layer of width m):* $f_{v,W}(x) := (Wx)^\top v$ where $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$.
- Objective: $\mathcal{L}(v, W) = \frac{1}{2}\|(WX)^\top v - y\|^2$

Introduction & Problem setup

Linear Networks

- *Deep Linear (1 hidden layer of width m):* $f_{v,W}(x) := (Wx)^\top v$ where $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$.
- Objective: $\mathcal{L}(v, W) = \frac{1}{2}\|(WX)^\top v - y\|^2$
- Gradient flow:

$$\dot{v}_t = -\nabla_v \mathcal{L}(v_t, W_t) = W_t X \big(y - X^\top W_t^\top v_t\big),$$

Introduction & Problem setup

Linear Networks

- *Deep Linear (1 hidden layer of width m):* $f_{v,W}(x) := (Wx)^\top v$ where $W \in \mathbb{R}^{m\times d}$, $v \in \mathbb{R}^m$.
- Objective: $\mathcal{L}(v, W) = \frac{1}{2}\|(WX)^\top v - y\|^2$
- Gradient flow:

$$\dot{v}_t = -\nabla_v \mathcal{L}(v_t, W_t) = W_t X\big(y - X^\top W_t^\top v_t\big),$$

$$\dot{W}_t = -\nabla_W \mathcal{L}(v_t, W_t) = v_t\big(y - X^\top W_t^\top v_t\big)^\top X^\top.$$

Introduction & Problem setup

Linear Networks

- *Deep Linear (1 hidden layer of width m):* $f_{v,W}(x) := (Wx)^\top v$ where $W \in \mathbb{R}^{m\times d}$, $v \in \mathbb{R}^m$.
- Objective: $\mathcal{L}(v, W) = \frac{1}{2}\|(WX)^\top v - y\|^2$
- Gradient flow:

$$\dot{v}_t = -\nabla_v \mathcal{L}(v_t, W_t) = W_t X\big(y - X^\top W_t^\top v_t\big),$$

$$\dot{W}_t = -\nabla_W \mathcal{L}(v_t, W_t) = v_t\big(y - X^\top W_t^\top v_t\big)^\top X^\top.$$

- Introducing depth already creates complex dynamics (coupled ODEs of order three) with no known general analytic solutions.

Introduction & Problem setup

Non-Linear Networks

- $f_{v,W}(X) = \phi(Wx)^\top v$ where $\phi : \mathbb{R} \to \mathbb{R}$ is a continuously differentiable (a.e.) function applied elementwise, called an activation function.
- Objective: $\mathcal{L}(v, W) = \frac{1}{2}\|\phi(WX)^\top v - y\|^2$

Introduction & Problem setup

Non-Linear Networks

- $f_{v,W}(X) = \phi(Wx)^\top v$ where $\phi : \mathbb{R} \to \mathbb{R}$ is a continuously differentiable (a.e.) function applied elementwise, called an activation function.
- Objective: $\mathcal{L}(v, W) = \frac{1}{2}\|\phi(WX)^\top v - y\|^2$
- Gradient flow:

$$\dot{v}_t = -\nabla_v \mathcal{L}(v_t, W_t) = \phi(W_t X)\big(y - X^\top W_t^\top v_t\big),$$

Introduction & Problem setup

Non-Linear Networks

- $f_{v,W}(X) = \phi(Wx)^\top v$ where $\phi : \mathbb{R} \to \mathbb{R}$ is a continuously differentiable (a.e.) function applied elementwise, called an activation function.
- Objective: $\mathcal{L}(v, W) = \frac{1}{2}\|\phi(WX)^\top v - y\|^2$
- Gradient flow:

$$\dot{v}_t = -\nabla_v \mathcal{L}(v_t, W_t) = \phi(W_t X)\big(y - X^\top W_t^\top v_t\big),$$

$$\dot{W}_t = -\nabla_W \mathcal{L}(v_t, W_t) = \Big(\phi'(W_t X) \odot \big(v_t\big(y - \phi(W_t X)^\top v_t\big)^\top\big)\Big) X^\top.$$

Introduction & Problem setup

Non-Linear Networks

- $f_{v,W}(X) = \phi(Wx)^\top v$ where $\phi : \mathbb{R} \to \mathbb{R}$ is a continuously differentiable (a.e.) function applied elementwise, called an activation function.
- Objective: $\mathcal{L}(v, W) = \frac{1}{2}\|\phi(WX)^\top v - y\|^2$
- Gradient flow:

$$\dot{v}_t = -\nabla_v \mathcal{L}(v_t, W_t) = \phi(W_t X)\big(y - X^\top W_t^\top v_t\big),$$

$$\dot{W}_t = -\nabla_W \mathcal{L}(v_t, W_t) = \Big(\phi'(W_t X) \odot \big(v_t\big(y - \phi(W_t X)^\top v_t\big)^\top\big)\Big) X^\top.$$

- Activation functions mix matrix products with elementwise operations, we don't know of mathematical tools to handle these kinds of expressions.

Why Analyze Training Dynamics?

- For linear models, gradient descent is a linear dynamical system governed by the spectrum of $XX^\top$ with a closed-form solution.

Why Analyze Training Dynamics?

- For linear models, gradient descent is a linear dynamical system governed by the spectrum of $XX^\top$ with a closed-form solution.
- For nonlinear networks, training dynamics are coupled and nonlinear, with no general closed-form description.

Why Analyze Training Dynamics?

- For linear models, gradient descent is a linear dynamical system governed by the spectrum of $XX^\top$ with a closed-form solution.
- For nonlinear networks, training dynamics are coupled and nonlinear, with no general closed-form description.
- But having analytical solutions allows understanding the behaviour of the system.

Why Analyze Training Dynamics?

- For linear models, gradient descent is a linear dynamical system governed by the spectrum of $XX^\top$ with a closed-form solution.
- For nonlinear networks, training dynamics are coupled and nonlinear, with no general closed-form description.
- But having analytical solutions allows understanding the behaviour of the system.
- This motivates studying **simplified regimes** where the dynamics become analytically tractable.

Why Analyze Training Dynamics?

- For linear models, gradient descent is a linear dynamical system governed by the spectrum of $XX^\top$ with a closed-form solution.
- For nonlinear networks, training dynamics are coupled and nonlinear, with no general closed-form description.
- But having analytical solutions allows understanding the behaviour of the system.
- This motivates studying **simplified regimes** where the dynamics become analytically tractable.
- Two prominent approaches: the **NTK** and the **mean-field** regime (we focus on the NTK).

The NTK Regime

Linearization at Infinite Width

- **Motivation:** To analyze training dynamics by linearizing the network around initialization.
- **Formalization:**
  - As width $m \to \infty$, the empirical kernel Θ_t converges to a deterministic, static limit $\bar{\Theta}$.

$$\bar{\Theta}(x, x') = \mathbb{E}_w\big[\varphi(w^\top x)\,\varphi(w^\top x')\big] + \sigma_v^2\, x^\top x'\, \mathbb{E}_w\big[\varphi'(w^\top x)\,\varphi'(w^\top x')\big]$$

  - The network function f_t evolves linearly with respect to this frozen kernel.
- **Training Dynamics:**
  - Equivalent to Kernel Ridge Regression with the NTK.
  - Convergence is guaranteed if the limit kernel is positive definite.

2b. Related Work: The Mean-Field View

Feature Learning at Infinite Width

- **Motivation:** To capture feature learning, which is absent in the "lazy" NTK regime.
- **Formalization:**
  - Weights are treated as particles drawn from a distribution μ.
  - Output is scaled by $1/n$ (vs $1/\sqrt{n}$ in NTK).

$$f_{v,w}(x) = \frac{1}{m}\sum_{\alpha=1}^{m} v_\alpha \varphi(w_\alpha^\top x) = \int_\Omega v\,\varphi(w^\top x)\,d\mu(v,w)$$

- **Training Dynamics:**
  - Modeled as a Wasserstein gradient flow of the probability measure μ_t.
  - Allows the kernel (and features) to evolve during training.

2c. Related Work: Spectral Bias

Frequency-Dependent Convergence

- **Phenomenon:** Neural networks fit low-frequency components of the target function faster than high-frequency noise.
- **Theoretical Basis:**
  - Convergence along eigen-directions is determined by the eigenvalues λ_k of the NTK integral operator.
  - High frequencies correspond to small eigenvalues $\rightarrow$ slow convergence.
- **Implications:** Provides a theoretical basis for early stopping and generalization in overparameterized models.

3. Preliminary Experiments

1. **Function Space Convergence**
2. **Kernel Drift & Regime Transitions**
3. **Spectral Analysis of Empirical NTK**

Summary & Discussion

Structure of Introduction & Literature Review:

1. **Foundations:** From linear regression to deep linear dynamics.
2. **Infinite Limits:** NTK limit (static kernel), Mean-Field limit (changing kernel).
3. **Spectral Properties:** How eigenvalues dictate learnability (Spectral Bias).

Question for Supervisors: Does this structure logically support the move to finite-width deviations in later chapters?